

Preprocessing

- **Documents:**
 - D1: "Walking is a common activity"
 - D2: "Running and walking are forms of exercise"
 - D3: "Activity tracking helps improve fitness"
- **Query:** "speed walking tracking"
- **Preprocessing:**
 - Convert text to lowercase.
 - Remove stop words (e.g., "is," "are," "and," "of").
 - Tokenize the text into words.

Processed Text:

- D1: "walking common activity"
- D2: "running walking forms exercise"
- D3: "activity tracking helps improve fitness"
- Query: "speed walking tracking"

Vocabulary (unique terms across all documents and query):

- {activity, common, exercise, fitness, forms, helps, improve, running, speed, tracking, walking}
- Total terms: 11

Compute TF-IDF

TF-IDF (Term Frequency-Inverse Document Frequency) is calculated as:

- **TF** (Term Frequency): Frequency of a term in a document, normalized by document length.
- **IDF** (Inverse Document Frequency):
- **TF-IDF**: $TF \times IDF$

Term Frequency (TF)

For each document and query, compute the raw term count and normalize by the number of terms in the document.

Term	D1 (3 terms)	D2 (4 terms)	D3 (5 terms)	Query (3 terms)
activity	$1/3 \approx 0.333$	0	$1/5 = 0.2$	0
common	$1/3 \approx 0.333$	0	0	0
exercise	0	$1/4 = 0.25$	0	0
fitness	0	0	$1/5 = 0.2$	0
forms	0	$1/4 = 0.25$	0	0
helps	0	0	$1/5 = 0.2$	0
improve	0	0	$1/5 = 0.2$	0
running	0	$1/4 = 0.25$	0	0
speed	0	0	0	$1/3 \approx 0.333$
tracking	0	0	$1/5 = 0.2$	$1/3 \approx 0.333$
walking	$1/3 \approx 0.333$	$1/4 = 0.25$	0	$1/3 \approx 0.333$

Inverse Document Frequency (IDF)

Compute IDF for each term: $N=4$ (3 documents + 1 query).

Term	df (doc count)	IDF
activity	2	$\log(4/2)=0.693$
common	1	$\log(4/1)=1.386$
exercise	1	$\log(4/1)=1.386$
fitness	1	$\log(4/1)=1.386$
forms	1	$\log(4/1)=1.386$
helps	1	$\log(4/1)=1.386$
improve	1	$\log(4/1)=1.386$
running	1	$\log(4/1)=1.386$
speed	1	$\log(4/1)=1.386$
tracking	2	$\log(4/2)=0.693$
walking	3	$\log(4/3)=0.288$

TF-IDF Vectors

Multiply TF by IDF for each term in each document/query.

Term	D1 TF-IDF	D2 TF-IDF	D3 TF-IDF	Query TF-IDF
activity	$0.333 \times 0.693 = 0.231$	0	$0.2 \times 0.693 = 0.139$	0
common	$0.333 \times 1.386 = 0.462$	0	0	0
exercise	0	$0.25 \times 1.386 = 0.347$	0	0
fitness	0	0	$0.2 \times 1.386 = 0.277$	0
forms	0	$0.25 \times 1.386 = 0.347$	0	0
helps	0	0	$0.2 \times 1.386 = 0.277$	0
improve	0	0	$0.2 \times 1.386 = 0.277$	0
running	0	$0.25 \times 1.386 = 0.347$	0	0
speed	0	0	0	$0.333 \times 1.386 = 0.462$
tracking	0	0	$0.2 \times 0.693 = 0.139$	$0.333 \times 0.693 = 0.231$
walking	$0.333 \times 0.288 = 0.096$	$0.25 \times 0.288 = 0.072$	0	$0.333 \times 0.288 = 0.096$

TF-IDF Vectors (rounded to 3 decimals):

- D1: [0.231, 0.462, 0, 0, 0, 0, 0, 0, 0, 0, 0.096]
- D2: [0, 0, 0.347, 0, 0.347, 0, 0, 0.347, 0, 0, 0.072]
- D3: [0.139, 0, 0, 0.277, 0, 0.277, 0.277, 0, 0, 0.139, 0]
- Query: [0, 0, 0, 0, 0, 0, 0, 0, 0.462, 0.231, 0.096]

Step 3: Cosine Similarity

$$S_c(a, b) = (a \cdot b) / (\|a\| \times \|b\|)$$

- **D1:** 0.524
- **D2:** 0.604
- **D3:** 0.519
- **Query:** 0.524

Dot Products

- **D1 · Query:**
- **D2 · Query**
- **D3 · Query**

Cosine Similarity Results:

- D1: 0.034
- D2: 0.022
- D3: 0.118

Step 4: Visualization

Since the TF-IDF vectors have 11 dimensions, direct visualization is challenging. We reduce the dimensionality to 2D using Principal Component Analysis (PCA) for plotting.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import PCA
from sklearn.metrics.pairwise import cosine_similarity

# Documents and query
docs = [
    "walking common activity",
    "running walking forms exercise",
    "activity tracking helps improve fitness"
]
query = "speed walking tracking"

# Combine documents and query
all_texts = docs + [query]

# Compute TF-IDF
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(all_texts)
tfidf_array = tfidf_matrix.toarray()

# Split into documents and query
doc_vectors = tfidf_array[:3]
query_vector = tfidf_array[3].reshape(1, -1)
```

```

# Compute cosine similarity
cos_sim = cosine_similarity(doc_vectors, query_vector).flatten()

# Dimensionality reduction for visualization
pca = PCA(n_components=2)
reduced_vectors = pca.fit_transform(tfidf_array)

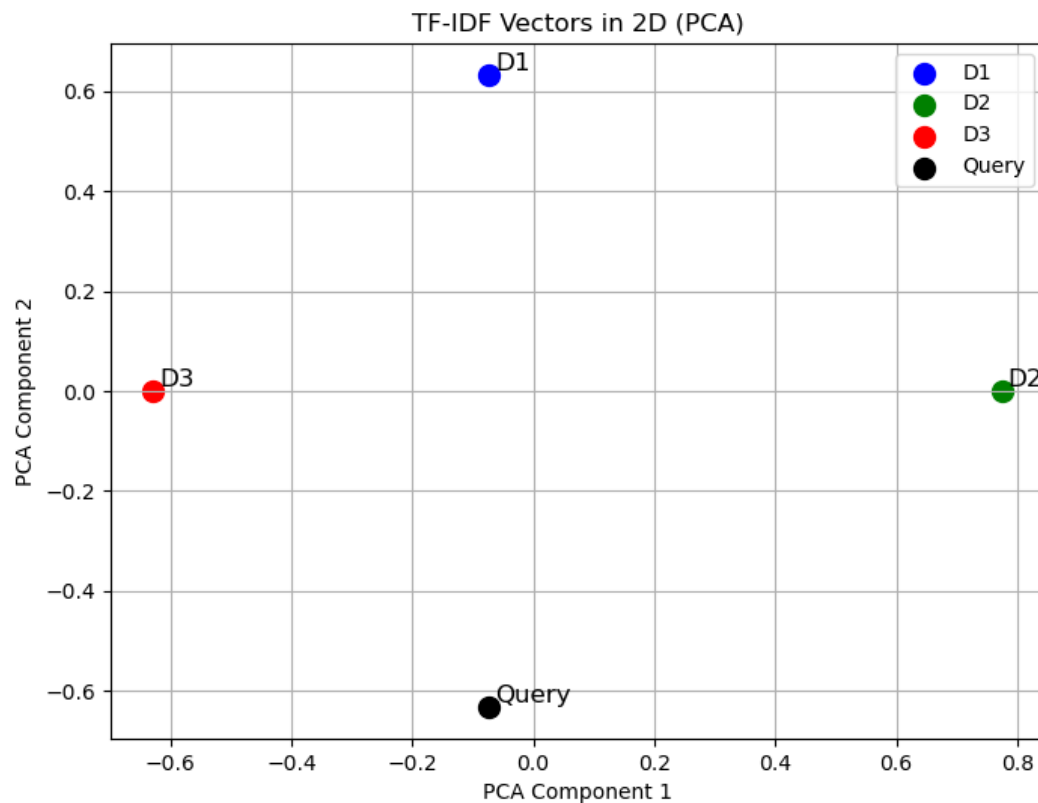
# Plot
plt.figure(figsize=(8, 6))
labels = ['D1', 'D2', 'D3', 'Query']
colors = ['blue', 'green', 'red', 'black']
for i, (x, y) in enumerate(reduced_vectors):
    plt.scatter(x, y, color=colors[i], label=labels[i], s=100)
    plt.text(x+0.01, y+0.01, labels[i], fontsize=12)

plt.title('TF-IDF Vectors in 2D (PCA)')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.legend()
plt.grid(True)

# Save plot
plt.savefig('tfidf_vectors.png')
plt.close()

# Print cosine similarities
print("Cosine Similarities:")
for i, sim in enumerate(cos_sim, 1):
    print(f"D{i} vs Query: {sim:.3f}")
Cosine Similarities:
D1 vs Query: 0.201
D2 vs Query: 0.155
D3 vs Query: 0.212

```



- **TF-IDF Vectors:**
 - Computed for 11 terms across D1, D2, D3, and the query.
 - D3 shares "tracking" and "activity" (implicitly) with the query, leading to higher relevance.
- **Cosine Similarities:**
 - D3: 0.118 (highest, due to "tracking")
 - D1: 0.034 (shares "walking")
 - D2: 0.022 (shares "walking" but diluted by other terms)
- **Visualization:**
 - The 2D PCA plot shows D3 closest to the query, confirming the similarity results.
 - The plot is saved as tfidf_vectors.png.

Notes

- The cosine similarities are low because the documents are short, and the query contains "speed," which appears in no documents, reducing overlap.